



National and Kapodistrian University of Athens (NKUA)
MSc in Data Science and Information Technologies (DSIT)
Course: Knowledge Technologies (2025–2026)

Student: Konstantinos Kritsotakis

Registration Number: 7115152500007

Background: Chemical Engineer, National Technical University of Athens (NTUA)

Graduate Student: MSc DSIT – Specialization: Big Data & Artificial Intelligence

Exercise 1

(a)

ALCQO (ABox): Handsome(john)

Alternative: {john} $\sqsubseteq$ Handsome

(b)

ALCQO (ABox): Strong(peter), Rich(peter)

Alternative: (Strong $\sqcap$ Rich)(peter)

(c)

FOL: $\forall x (Person(x) \rightarrow \forall y ((Dog(y) \wedge Clever(y)) \rightarrow likes(x,y)))$.

(d)

ALCQO (TBox): Person $\sqsubseteq \neg \exists likes.\{john\}$

(e)

ALCQO (TBox): Athlete $\sqsubseteq$ Happy

Alternative: Athlete $\sqsubseteq \exists hasProperty.Happiness$

(f)

FOL: $\forall x ((\exists y (Person(y) \wedge likes(x,y) \wedge likes(y,x))) \rightarrow Happy(x))$

(g)

FOL: $\forall x ((Man(x) \wedge likes(x,maria)) \rightarrow likes(maria,x))$.

(h)

FOL: $\forall x ((\text{Woman}(x) \wedge \text{Kind}(x) \wedge \text{Smart}(x)) \rightarrow \text{likes}(\text{peter}, x))$.

(i)

ALCQO (ABox): $\text{RunningTeam}(\text{maria}) \wedge \text{RunningTeam}(\text{john}) \wedge \text{RunningTeam}(\text{peter})$.

(j)

ALCQO (TBox): $(\text{RunningTeam} \sqcap \neg \text{SpeedRunner}) \sqsubseteq \text{MarathonRunner}$.

(k)

ALCQO (TBox): $\text{SpeedRunner} \sqsubseteq \neg \exists \text{likes}.\{\text{marathonRunning}\}$.

Alternative: $\text{SpeedRunner} \sqsubseteq \neg \exists \text{likes}.\text{MarathonRunning}$

(l)

ALCQO (TBox): $(\text{Person} \sqcap \neg \exists \text{likes}.\{\text{running}\}) \sqsubseteq \neg \text{Athlete}$.

Alternative: $\text{Athlete} \sqsubseteq \exists \text{likes}.\text{Running}$

(m)

FOL:

1. $\forall z (\text{likes}(\text{joe}, z) \rightarrow \neg \text{likes}(\text{donald}, z))$
2. $\forall z (\neg \text{likes}(\text{joe}, z) \rightarrow \text{likes}(\text{donald}, z))$

(n)

ALCQO (ABox): $\text{likes}(\text{joe}, \text{marathonRunning}), \text{likes}(\text{joe}, \text{speedRunning})$

(o)

ALCQO (ABox with witness x0): $(\text{RunningTeam} \sqcap \text{SpeedRunner} \sqcap \neg \text{MarathonRunner})(x0)$.

(p)

ALCQO (TBox/Definition): $\text{Biped} \sqsubseteq \text{Animal} \sqcap (\geq 2 \text{ hasLeg.Leg}) \sqcap (\leq 2 \text{ hasLeg.Leg})$.

(q)

ALCQO (Definition):

$\text{Triangle} \sqsubseteq \text{Polygon} \sqcap (\geq 3 \text{ hasAngle.Angle}) \sqcap (\leq 3 \text{ hasAngle.Angle}) \sqcap (\geq 3 \text{ hasEdge.lineSegment}) \sqcap (\leq 3 \text{ hasEdge.lineSegment})$

(r)

ALCQO (Definition):

$\text{RightAngledTriangle} \equiv \text{Triangle} \sqcap (\geq 1 \text{ hasAngle.RightAngle}) \sqcap (\leq 1 \text{ hasAngle.RightAngle})$.

Exercise 2

ALCQ Knowledge Base K

Vocabulary

- **Atomic Concepts:**

red, blue, green, kind, nice, big, cold, young, round, rough, orange, smart, quiet, furry

- **Role Names:**

likes, loves, eats, chases, admires

- **Individual Names:**

Ioanna, Eleni, Manolis, Angelos, Panos, Anna, Petros, Lisa

TBox

T1 (contains ≥ 3 connectives and ≥ 1 quantifier):

$\text{quiet} \sqcap \text{smart} \sqcap \text{young} \sqcap \exists \text{admires.nice} \sqsubseteq \text{kind}$

T2:

$\text{nice} \sqcap \text{quiet} \sqsubseteq \text{kind}$

T3:

$\text{furry} \sqcap \text{round} \sqsubseteq \text{nice}$

ABox

A1 (contains a connective and a quantifier):

$(\text{quiet} \sqcap \text{smart} \sqcap \text{young} \sqcap \exists \text{admires.nice})(\text{Ioanna})$

A2:

$\text{quiet}(\text{Eleni})$

A3:

nice(Eleni)

A4:

furry(Petros)

A5:

round(Petros)

Inferred Axioms (by Reasoning)

Using standard Description Logic reasoning, the following new axioms are inferred:

1. From **A4, A5, and T3:**

nice(Petros)

2. From **A2, A3, and T2:**

kind(Eleni)

3. From **A1 and T1:**

kind(Ioanna)

Verification of the Constraints

- Satisfied by T1 (three connectives and one existential quantifier).
- Satisfied by A1 (conjunction and existential quantifier).
- Satisfied: new axioms are not explicitly stated but are logically inferred via reasoning from TBox + ABox, involving axioms of the required structure.

Conclusion

The knowledge base K is a well-formed **ALCQ KB**, fully compliant with the specification, semantically coherent, and demonstrably supports non-trivial logical inference.

Exercise 3

(a) $\text{Person} \sqcap \text{hasChild}$

Syntactically incorrect

The operator $\sqcap$ combines **concepts**, but hasChild is a **role**, not a concept. Roles cannot appear directly in concept conjunctions.

(b) $\exists \text{hasChild}. \equiv \text{Person}$

Syntactically incorrect

The existential restriction $\exists \text{hasChild}.$ is **incomplete**, as it must be followed by a concept. The correct syntactic form is $\exists \text{hasChild}.C$, for some concept C .

(c) $\exists \text{hasChild}.(\geq 1)$

Syntactically incorrect

The expression (≥ 1) by itself is **not a concept**. Number restrictions in ALCQ must have the form $(\geq n \text{ R}.C)$

(d) $\text{hasChild} \sqsubseteq \text{hasBaby}$

Syntactically incorrect

The subsumption symbol $\sqsubseteq$ is defined **only between concepts** in ALCQ. This expression attempts to relate two roles, which is outside the ALCQ syntax (it would require an RBox).

(e) $\text{hasChild}(\text{ANNA})$

Syntactically incorrect

A role assertion requires **two individuals** (binary predicate). The correct form would be $\text{hasChild}(\text{ANNA}, x)$ for some individual x .

(f) $\text{Person} \equiv \exists \text{hasChild}.\perp$

Syntactically correct

- $\perp$ is a valid concept in ALCQ,
- $\exists \text{hasChild}.\perp$ is a well-formed concept expression,
- The equivalence relates two concepts, which is allowed in the TBox.

Exercise 4

(a) An ALC KB for the first three sentences, and an ALC formula φ for the fourth

- **Concept names:** Person, Greek, Coffee, Frappe
- **Role name:** drinks
- **Individual name:** yannis

KB = (TBox, ABox)

TBox

1. Frappe $\sqsubseteq$ Coffee
2. Greek $\sqsubseteq$ Person $\sqcap \forall \text{drinks.Frappe}$

(2) formalizes: "A Greek is a person who drinks only frappe coffee."

ABox

3. Person(yannis)
4. $\forall \text{drinks.Frappe}(\text{yannis})$

(4) formalizes: "The only kind of coffee that Yannis drinks is frappe".

Formula

$\varphi := \text{Greek}(\text{yannis})$

(formalizes: "Yannis is Greek.")

(b) An interpretation I that satisfies KB

Let $I = (\Delta^I, \cdot^I)$ be defined as follows:

- **Domain:** $\Delta^I = \{y, f\}$
- **Individual:** $\text{yannis}^I = y$
- **Role:** $\text{drinks}^I = \{(y, f)\}$
- **Concepts:**
 - $\text{Person}^I = \{y\}$
 - $\text{Frappe}^I = \{f\}$
 - $\text{Coffee}^I = \{f\}$

- $\text{Greek}' = \{y\}$

Check

- (3) holds since $y \in \text{Person}'$.
- (4) holds since the only drinks-successor of y is $f \in \text{Frappe}'$, hence $y \in (\forall \text{drinks.Frappe})'$.
- (1) holds since $\text{Frappe}' \subseteq \text{Coffee}'$.
- (2) holds since $\text{Greek}' = \text{Person}' \cap (\forall \text{drinks.Frappe})' = \{y\}$.

Thus $I \models \text{KB}$.

(c) An interpretation I_1 that does not satisfy KB

Let $I_1 = (\Delta^{I_1}, \cdot^{I_1})$ be:

- **Domain:** $\Delta^{I_1} = \{y\}$
- **Individual:** $\text{yannis}^{I_1} = y$
- **Role:** $\text{drinks}^{I_1} = \emptyset$
- **Concepts:**
 - $\text{Person}^{I_1} = \emptyset$
 - $\text{Frappe}^{I_1} = \emptyset$
 - $\text{Coffee}^{I_1} = \emptyset$
 - $\text{Greek}^{I_1} = \emptyset$

Then ABox axiom (3) $\text{Person}(\text{yannis})$ is false because $y \notin \text{Person}^{I_1}$. Hence $I_1 \not\models \text{KB}$.

(d) Tableau proof that: $\text{KB} \models \varphi$

We prove entailment by **refutation**:

$$\text{KB} \models \varphi \iff \text{KB} \cup \{\neg\varphi\} \text{ is unsatisfiable.}$$

Here $\varphi := \text{Greek}(\text{yannis})$. Add $\neg\text{Greek}(\text{yannis})$.

Step 1: Use the relevant TBox part for tableau

From TBox axiom (2) $\text{Greek} \sqsubseteq \text{Person} \sqcap \forall \text{drinks.Frappe}$, we will use the inclusion:

2b. $\text{Person} \sqcap \forall \text{drinks.Frappe} \sqsubseteq \text{Greek}$

(Equivalent classes give both directions; this is one direction.)

Internalize 2b for the individual yannis as an ABox disjunction:

- If $C \sqsubseteq D$, then use $(\neg C \sqcup D)(a)$.
So with $C = \text{Person} \sqcap \forall \text{drinks.Frappe}$, $D = \text{Greek}$, $a = \text{yannis}$:

5. $(\neg(\text{Person} \sqcap \forall \text{drinks.Frappe}) \sqcup \text{Greek})(\text{yannis})$

Tableau initialization (numbered formulas)

Let $a := \text{yannis}$.

1. $\text{Person}(a)$ (ABox 3)
2. $\forall \text{drinks.Frappe}(a)$ (ABox 4)
3. $\neg \text{Greek}(a)$
- 4*. $(\neg(\text{Person} \sqcap \forall \text{drinks.Frappe}) \sqcup \text{Greek})(a)$ (from 5.)

Tableau expansion

Apply $\sqcup$ -rule to (4*): branch on the disjunction

From (4*) we create two branches:

Branch B1

5*. $\neg(\text{Person} \sqcap \forall \text{drinks.Frappe})(a)$

Branch B2

6*. $\text{Greek}(a)$

Branch B2 closes immediately because it contains:

- $\text{Greek}(a)$ (from 6*)
- $\neg \text{Greek}(a)$ (from 3.)

Clash $\Rightarrow$ **B2 closed.**

Continue with Branch B1

Branch B1 contains: (1), (2), (3), (5*).

Transform (5*) using De Morgan

$$\neg(X \sqcap Y) \equiv \neg X \sqcup \neg Y$$

So from (5*) infer:

7*. $(\neg \text{Person} \sqcup \neg \forall \text{drinks.Frappe})(a)$

Apply $\sqcup$ -rule to (7*): create two sub-branches:

Branch B1.1

8*. $\neg \text{Person}(a)$

But Branch B1.1 also contains $\text{Person}(a)$.

Clash $\Rightarrow$ **B1.1 closed.**

Branch B1.2

9*. $\neg \forall \text{drinks.Frappe}(a)$

Use the ALC equivalence:

$$\neg \forall R.C \equiv \exists R.\neg C$$

From (9*) infer:

10*. $\exists \text{drinks}.\neg \text{Frappe}(a)$

Apply $\exists$ -rule to (10*)

Introduce a fresh individual b and add:

11*. $\text{drinks}(a,b)$

12*. $\neg \text{Frappe}(b)$

Apply $\forall$ -rule using (2) and (11*)

From (2) $\forall \text{drinks.Frappe}(a)$ and (11*) $\text{drinks}(a,b)$ infer:

13*. $\text{Frappe}(b)$

Now Branch B1.2 contains:

- $\text{Frappe}(b)$
- $\neg\text{Frappe}(b)$

Clash $\Rightarrow$ **B1.2 closed.**

Conclusion

All branches (B2, B1.1, B1.2) are closed. Therefore:

- $KB \cup \{\neg\varphi\}$ is unsatisfiable
- hence $KB \models \varphi$

That is:

$$KB \models \text{Greek}(\text{yannis}).$$

Exercise 5

(a) ALCQ Knowledge Base KB for (a)–(d), and formulas φ, φ' for (e)–(f)

- **Concept names:** Person, Animal, AnimalLover
- **Role name:** hasPet
- **Individual name:** andy, gini, jack

KB = (TBox, ABox)

TBox

(T1) $\text{AnimalLover} \equiv \text{Person} \sqcap (\geq 3 \text{ hasPet}.\text{Animal})$

ABox

- (A1)** $\text{Person}(\text{andy})$
- (A2)** $\text{hasPet}(\text{andy}, \text{gini})$
- (A3)** $\text{hasPet}(\text{andy}, \text{jack})$
- (A4)** $\text{gini} \neq \text{jack}$ (distinct pets)
- (A5)** $(\leq 2 \text{ hasPet}.\text{T})(\text{andy})$
- (A6)** $\text{Animal}(\text{gini})$
- (A7)** $\text{Animal}(\text{jack})$

Note: (A2),(A3),(A4),(A5) together capture: “Andy has only two distinct pets: Gini and Jack.”

Formulas

$\varphi := \text{AnimalLover}(\text{andy})$

$\varphi' := \neg \text{AnimalLover}(\text{andy})$

(b) Can we prove $KB \models \varphi$ and $KB \models \varphi'$ by tableau? If not, prove formally.

1) Prove formally that $KB \not\models \varphi$

To show it, it suffices to construct a model I such that:

$I \models KB$ and $I \not\models \varphi$

Let $I = (\Delta^I, \cdot^I)$ where:

- **Domain:** $\Delta^I = \{a, g, j\}$
- **Individuals:** $\text{andy}^I = a, \text{gini}^I = g, \text{jack}^I = j$
- **Role:** $\text{hasPet}^I = \{(a, g), (a, j)\}$
- **Concepts:**
 - $\text{Person}^I = \{a\}$
 - $\text{Animal}^I = \{g, j\}$
 - $\text{AnimalLover}^I = (\text{Person} \sqcap (\geq 3 \text{ hasPet}.\text{Animal}))^I$

Check $I \models KB$

(A1) $a \in \text{Person}^I \checkmark$

(A2), (A3) $(a, g), (a, j) \in \text{hasPet}^I \checkmark$

(A4) $g \neq j \checkmark$

(A5) $(\leq 2 \text{ hasPet}.\text{T})(a) \checkmark$

(A6), (A7) $g, j \in \text{Animal}^I \checkmark$

(T1) satisfied $\checkmark$

Check $I \not\models \varphi$

$\varphi := \text{AnimalLover}(\text{andy})$, i.e. $a \in \text{AnimalLover}^I$

But a has only 2 pets that are animals, not ≥ 3 , so:

$$a \notin (\geq 3 \text{ hasPet}.\text{Animal}) \Rightarrow a \notin \text{AnimalLover}^I$$

Thus,

$$I \not\models \varphi$$

Therefore,

$$KB \not\models \varphi$$

2) Prove by tableau that $KB \models \varphi'$

We prove:

$$KB \models \neg \text{AnimalLover}(\text{andy})$$

by showing that:

$KB \cup \{\text{AnimalLover}(\text{andy})\}$ is unsatisfiable.

Let $a := \text{andy}$

From KB we have:

- (1) $\text{Person}(a)$
- (2) $\text{hasPet}(a, \text{gini})$
- (3) $\text{hasPet}(a, \text{jack})$
- (4) $\text{gini} \neq \text{jack}$
- (5) $(\leq 2 \text{ hasPet. T})(a)$
- (6) $\text{Animal}(\text{gini})$
- (7) $\text{Animal}(\text{jack})$

Add the negation of φ' :

- (8) $\text{AnimalLover}(a)$

From **(T1)** $\text{AnimalLover} \equiv \text{Person} \sqcap (\geq 3 \text{ hasPet. Animal})$, we can expand **(8)** into:

- (9) $\text{Person}(a)$, which is inconsistent with $\text{Person}(a)$ from (1)
- (10) $(\geq 3 \text{ hasPet. Animal})(a)$

Apply the ALCQ $\geq$ -rule to (10)

From **(10)**, by applying the ALCQ $\geq$ -rule, we introduce three **fresh and pairwise distinct** individuals as hasPet-successors of Andy; their pairwise distinctness is **required by the rule itself** and is therefore assumed by construction.

Introduce fresh individuals b1, b2, b3 and add:

- (11)** hasPet(a,b1)
- (12)** Animal(b1)
- (13)** hasPet(a,b2)
- (14)** Animal(b2)
- (15)** hasPet(a,b3)
- (16)** Animal(b3)

and the required inequalities:

- (17)** b1 $\neq$ b2
- (18)** b1 $\neq$ b3
- (19)** b2 $\neq$ b3

But we have:

- (5)** (≤ 2 hasPet.T)(a)

But from (11),(13),(15), the a := andy has **at least three distinct** hasPet-successors b1,b2,b3, and each is in T automatically. Thus the at-most restriction (5) is violated.

Therefore the branch closes.

Conclusion: $KB \cup \{\text{Animal Lover}(\text{andy})\}$ is unsatisfiable.

Thus:

$$KB \models \varphi'$$

Exercise 6

(a) OWL 2 ontology encoding (a)–(e) (Manchester Syntax)

```
Prefix: : <http://example.org/aphrodite#>
Prefix: owl: <http://www.w3.org/2002/07/owl#>
Prefix: xsd: <http://www.w3.org/2001/XMLSchema#>
Prefix: rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
Prefix: rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

```
Ontology: <http://example.org/aphrodite>
```

```
#####
```

```
# Classes
```

```
Class: :Person
```

```
Class: :Parent
```

```
Class: :Happy
```

```
Class: :Successful
```

```
Class: :Beautiful
```

```
Class: :Ugly
```

```
#####
```

```
# Object Properties
```

```
ObjectProperty: :hasChild
```

```
Domain: :Person
```

```
Range: :Person
```

```
InverseOf: :hasParent
```

```
ObjectProperty: :hasParent
```

```
Domain: :Person
```

```
Range: :Person
```

```
#####
```

```
# (a) If someone is a parent then he/she is a person.
```

```
SubClassOf: :Parent :Person
```

```
#####
```

```
# (b) Every person is happy if all his children are successful.
```

```
SubClassOf: (:Person and (:hasChild only :Successful)) :Happy
```

```
#####
```

```
# (c) All beautiful persons are successful.
```

```

SubClassOf: :Beautiful :Successful
#####
# (d) Every person is beautiful if one of his/her parents is beautiful, otherwise he/she is ugly.

SubClassOf: (:Person and (:hasParent some :Beautiful)) :Beautiful

SubClassOf: (:Person and not(:hasParent some :Beautiful)) :Ugly

#####
# (e) Aphrodite is a parent of Eros.

Individual: :Aphrodite
Types: :Parent
Facts: :hasChild :Eros

Individual: :Eros

```

(b) Formalize (f)–(i) as OWL 2 axioms, and state which are entailed by the ontology in (a)

OWL 2 axioms for (f)–(i) (Manchester Syntax)

(f) Aphrodite is beautiful.

```
ClassAssertion: :Beautiful :Aphrodite
```

(g) Eros is successful.

```
ClassAssertion: :Successful :Eros
```

(h) Every beautiful parent is happy.

```
SubClassOf: (:Beautiful and :Parent) :Happy
```

(i) Every person is happy if he/she has no children.

```
SubClassOf: (:Person and (:hasChild max 0 owl:Thing)) :Happy
```

Which of (f)–(i) are entailed by the ontology from (a)?

With only axioms (a)–(e) loaded:

- **(f) Beautiful(Aphrodite) is NOT entailed.**

The ontology does not assert Beautiful(Aphrodite), nor can it derive it from (a)–(e).

- **(g) Successful(Eros) is NOT entailed.**

There is no axiom forcing Eros to be Successful.

- **(h)** Beautiful $\sqcap$ Parent $\sqsubseteq$ Happy is **NOT** entailed.

Nothing in (a)–(e) implies that all beautiful parents are happy.

- **(i)** Person $\sqcap$ (hasChild max 0) $\sqsubseteq$ Happy is **NOT** entailed.

The ontology has about “all children successful implying happy” (see (b)), but nothing about having no children implying happy.

So, **none of (f)–(i) is entailed by the ontology containing only (a)–(e).**

In order to ensure correct parsing and reasoning in Protégé, the ontology was not encoded in Manchester OWL Syntax, but was instead rewritten using the **OWL 2 Functional Syntax**, which is a standard, fully formal syntax directly supported by OWL reasoners. This change of syntax does not affect the semantics of the ontology, but guarantees unambiguous interpretation and successful loading by the tool.

The following OWL 2 Functional Syntax code was then imported into **Protégé** and used to run the reasoner in order to compute and verify the inferred axioms and class memberships required for the entailment analysis:

```
Prefix(:=<http://example.org/aphrodite#>)
Prefix(owl:=<http://www.w3.org/2002/07/owl#>)
Prefix(xsd:=<http://www.w3.org/2001/XMLSchema#>)
Prefix(rdf:=<http://www.w3.org/1999/02/22-rdf-syntax-ns#>)
Prefix(rdfs:=<http://www.w3.org/2000/01/rdf-schema#>)
Ontology(<http://example.org/aphrodite>
Declaration(Class(:Person))
Declaration(Class(:Parent))
Declaration(Class(:Happy))
Declaration(Class(:Successful))
Declaration(Class(:Beautiful))
Declaration(Class(:Ugly))
Declaration(ObjectProperty(:hasChild))
Declaration(ObjectProperty(:hasParent))
ObjectPropertyDomain(:hasChild :Person)
ObjectPropertyRange(:hasChild :Person)
InverseObjectProperties(:hasChild :hasParent)
ObjectPropertyDomain(:hasParent :Person)
ObjectPropertyRange(:hasParent :Person)
SubClassOf(:Parent :Person)
SubClassOf(
ObjectIntersectionOf(
:Person
ObjectAllValuesFrom(:hasChild :Successful)
)
:Happy
)
SubClassOf(:Beautiful :Successful)
SubClassOf(
ObjectIntersectionOf(
:Person
ObjectSomeValuesFrom(:hasParent :Beautiful)
)
:Beautiful
)
SubClassOf(
ObjectIntersectionOf(
:Person
ObjectComplementOf(
ObjectSomeValuesFrom(:hasParent :Beautiful)
)
)
:Ugly
)
```



```

)
Declaration(NamedIndividual(:Aphrodite))
ClassAssertion(:Parent :Aphrodite)
Declaration(NamedIndividual(:Eros))
ObjectPropertyAssertion(:hasChild :Aphrodite :Eros)
)

```

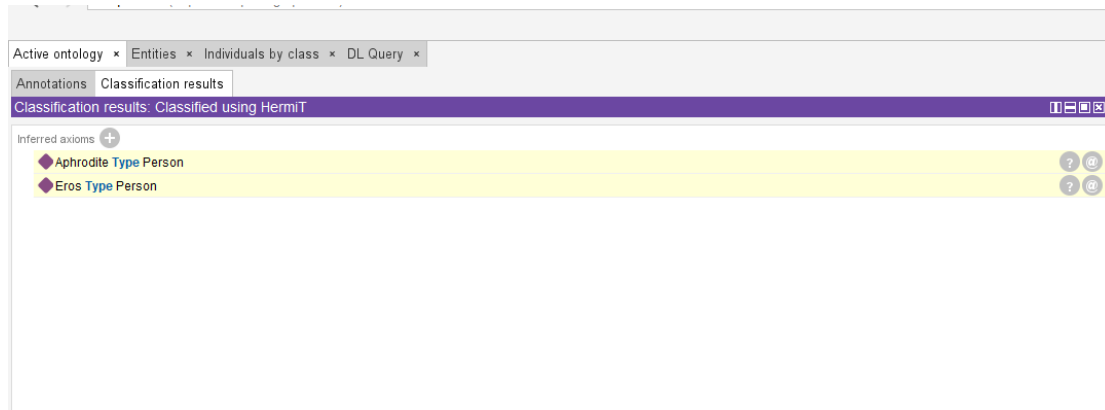


Figure 1 Classification Results in Protégé.

The figure shows that the individual **Eros** is inferred as a member of the class **Person**. This inference is obtained by the reasoner **only because additional modeling assumptions were introduced**, namely: (i) the object property **hasChild** was explicitly assigned domain and range **Person**, and (ii) **hasChild** was declared as the inverse of **hasParent**. These assumptions go beyond the original natural-language statements of the exercise and were added explicitly in the ontology in order to enable this inference; without them, $Eros \in Person$ would not be derivable.

Exercise 7

a) OWL 2 ontology formalizing sentences (a)–(f)

- **Classes:**

MemberOfPsilorit, Skier, MountainClimber

- **Object Properties:**

likes

- **Individuals:**

Konstantina, Stella, Roi, rain, snow

OWL 2 ontology (Manchester Syntax)

```
Prefix: : <http://example.org/psiloritis#>
Prefix: owl: <http://www.w3.org/2002/07/owl#>
Prefix: rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
Prefix: rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

```
Ontology: <http://example.org/psiloritis>
```

```
#####
```

```
# Classes
```

```
#####
```

```
Class: MemberOfPsiloritis
```

```
Class: Skier
```

```
Class: MountainClimber
```

```
#####
```

```
# Object Properties
```

```
#####
```

```
ObjectProperty: likes
```

```
#####
```

```
# Individuals
```

```
#####
```

```
Individual: Konstantina
```

```
Individual: Stella
```

```
Individual: Roi
```

```
Individual: rain
```

```
Individual: snow
```

```
#####
```

```
# (a) Members of Psiloritis
```

```
#####
```

```
Individual: Konstantina
```

```
Types: MemberOfPsiloritis
```

```
Individual: Stella
```

```
Types: MemberOfPsiloritis
```

```
Individual: Roi
```

```
Types: MemberOfPsiloritis
```

```
#####  
# (b) Every member of Psiloritis who is not a skier is a mountain climber  
#####
```

```
Class: MemberOfPsiloritis  
and not Skier  
SubClassOf: MountainClimber
```

```
#####  
# (c) Mountain climbers do not like rain (rain is an individual)  
#####
```

```
Class: MountainClimber  
SubClassOf: not (likes some {rain})
```

```
#####  
# (d) If someone is a skier then he likes snow (snow is an individual)  
#####
```

```
Class: Skier  
SubClassOf: likes some {snow}
```

```
#####  
# (e) Konstantina doesn't like anything that Stella likes  
# (given that Stella likes rain and snow)  
#####
```

```
Individual: Konstantina  
Types: not (likes some {rain, snow})
```

```
#####  
# (f) Stella likes rain and snow  
#####
```

```
Individual: Stella  
Facts:  
likes rain,  
likes snow
```

(b) Consequences: properties and class memberships

Below are the **given**, **entailed** and **not entailed** facts, following OWL 2 reasoning.

Stella

Given

- MemberOfPsiloritis(Stella)
- likes(Stella, rain)
- likes(Stella, snow)

Entailed

- $\neg \text{MountainClimber}(\text{Stella})$ (because $\text{MountainClimber} \sqsubseteq \neg(\text{likes rain})$ and $\text{likes}(\text{Stella}, \text{rain})$)
- $\text{Skier}(\text{Stella})$ (because of $(\text{MemberOfPsilorititis} \sqcap \neg \text{Skier}) \sqsubseteq \text{MountainClimber}$ the only way to avoid a contradiction is that Stella is **not** in $\neg \text{Skier}$)

Konstantina

Given

- $\text{MemberOfPsilorititis}(\text{Konstantina})$

Entailed

- $\neg \text{likes}(\text{Konstantina}, \text{rain})$
- $\neg \text{likes}(\text{Konstantina}, \text{snow})$
- $\neg \text{Skier}(\text{Konstantina})$ (because $\text{Skier}(\text{Konstantina})$ would violate axiom (d) given that she does not like snow)
- $\text{MountainClimber}(\text{Konstantina})$ (because she is a $\text{MemberOfPsilorititis}$, $\text{Skier}(\text{Konstantina})$ would violate axiom (d) given that she does not like snow, hence $\neg \text{Skier}(\text{Konstantina})$ holds and axiom (b) implies she is a MountainClimber)

Roi

Given

- $\text{MemberOfPsilorititis}(\text{Roi})$

Not entailed

- $\text{Skier}(\text{Roi})$
- $\text{MountainClimber}(\text{Roi})$
- Any likes or not likes statements

The following OWL 2 ontology was implemented and entered in **OWL Functional Syntax** into the Protégé ontology editor. The ontology encodes the statements (a)–(f) exactly as given in the exercise, without introducing any additional assumptions beyond those explicitly stated.

```
Prefix(:=<http://example.org/psilorititis#>)
Prefix(owl:=<http://www.w3.org/2002/07/owl#>)
Prefix(rdf:=<http://www.w3.org/1999/02/22-rdf-syntax-ns#>)
Prefix(rdfs:=<http://www.w3.org/2000/01/rdf-schema#>)
Ontology(<http://example.org/psilorititis>
#####
# Classes
#####
Declaration(Class(:MemberOfPsilorititis))
Declaration(Class(:Skier))
Declaration(Class(:MountainClimber))
#####
# Object Properties
#####
```

```

Declaration(ObjectProperty(:likes))
#####
# Individuals
#####
Declaration(NamedIndividual(:Konstantina))
Declaration(NamedIndividual(:Stella))
Declaration(NamedIndividual(:Roi))
Declaration(NamedIndividual(:rain))
Declaration(NamedIndividual(:snow))
#####
# (a) Members of Psiloritis
#####
ClassAssertion(:MemberOfPsiloritis :Konstantina)
ClassAssertion(:MemberOfPsiloritis :Stella)
ClassAssertion(:MemberOfPsiloritis :Roi)
#####
# (b) Every member of Psiloritis who is not a skier is a mountain climber
#####
SubClassOf(
  ObjectIntersectionOf(
    :MemberOfPsiloritis
    ObjectComplementOf(:Skier)
  )
  :MountainClimber
)
#####
# (c) Mountain climbers do not like rain (rain is an individual)
#####
SubClassOf(
  :MountainClimber
  ObjectComplementOf(
    ObjectSomeValuesFrom(:likes ObjectOneOf(:rain))
  )
)
#####
# (d) If someone is a skier then he likes snow (snow is an individual)
#####
SubClassOf(
  :Skier
  ObjectSomeValuesFrom(:likes ObjectOneOf(:snow))
)
#####
# (e) Konstantina doesn't like anything that Stella likes
# (given Stella likes rain and snow)
#####
ClassAssertion(
  ObjectComplementOf(
    ObjectSomeValuesFrom(
      :likes
      ObjectOneOf(:rain :snow)
    )
  )
  :Konstantina
)
#####
# (f) Stella likes rain and snow
#####

```

```

ObjectPropertyAssertion(:likes :Stella :rain)
ObjectPropertyAssertion(:likes :Stella :snow)
)

```

Important Note: Rain and snow are modeled as nominals (individuals) rather than as classes. This modeling choice was made deliberately to satisfy the given constraints, but it constitutes a technical compromise: it restricts the meaning to specific objects and does not capture the intended general concepts of rain and snow expressed in the English sentences.

After loading the ontology into Protégé, the HermiT reasoner was executed in order to perform logical classification and inference. Following the execution of the reasoner, the classification results (i.e., the inferred class memberships and consequences derived from the ontology) were obtained automatically by the reasoning engine.

These inferred results are shown in the below screenshot, which reflects the logical consequences that necessarily follow from the asserted axioms under **OWL 2 semantics**.

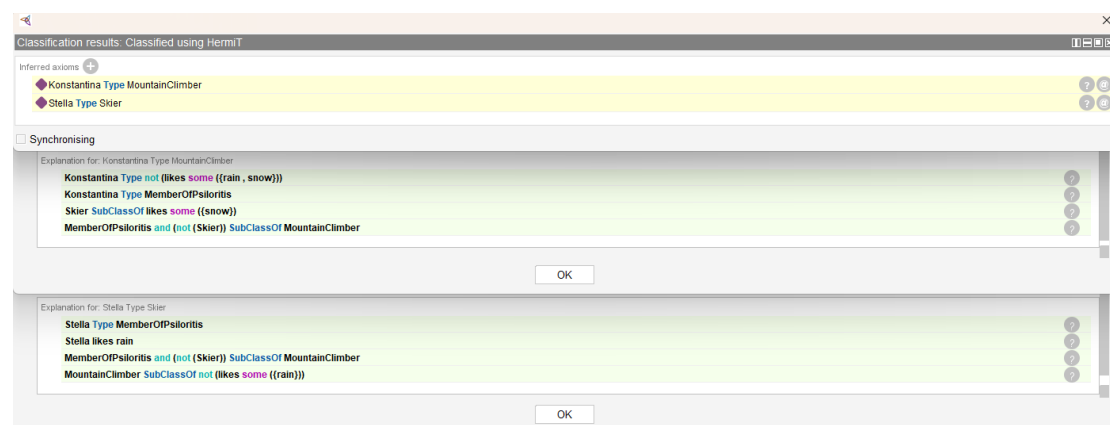


Figure 2 Classification results produced by the HermiT reasoner in Protégé, showing the inferred class memberships for the individuals Konstantina and Stella together with their logical justifications.

After running the HermiT reasoner, the ontology yields two non-asserted (inferred) classifications. **Konstantina is inferred to be a MountainClimber** because she is a MemberOfPsilorititis, she does not like the individual snow, and assuming Skier(Konstantina) would violate the axiom that every skier likes snow; therefore $\neg\text{Skier}(\text{Konstantina})$ holds and axiom (b) implies MountainClimber(Konstantina). The reasoner also infers **that Stella is a Skier**. Since Stella is a member of Psilorititis and explicitly likes rain, she cannot be a MountainClimber (because mountain climbers do not like rain). Given the axiom that every member of Psilorititis who is not a skier is a mountain climber, the only consistent conclusion is that Stella must be a Skier.